Challenge Name: Special Delivery

Category: REV

Description:

Special delivery! Let's see if you can find the flag now…

Hint(s):

If you've progressed a little bit and are struggling to find a direction to go, think about what behavior you know this program exhibits.

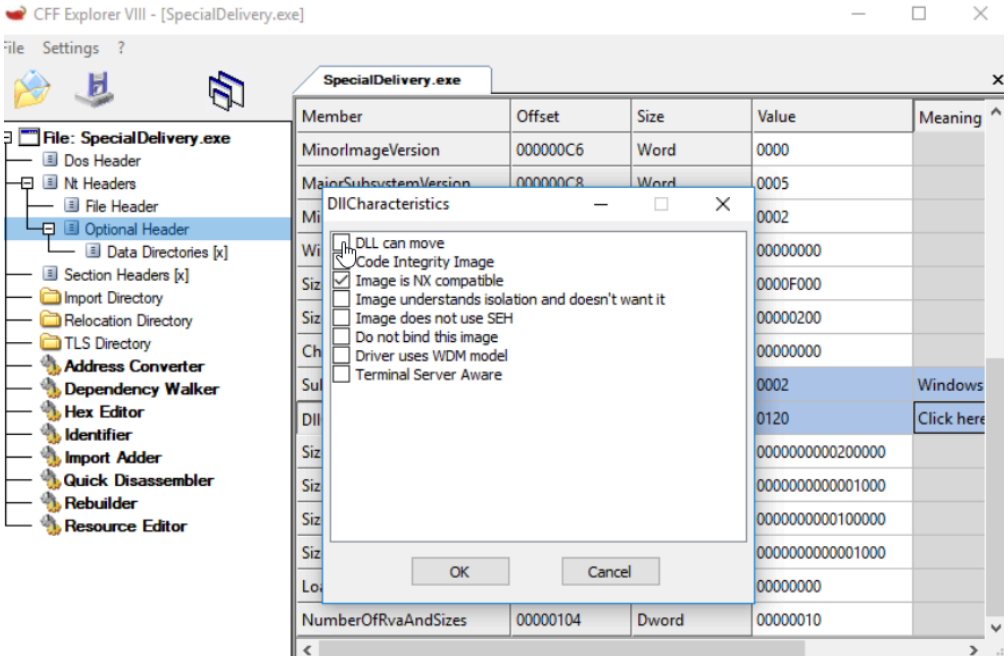
Walkthrough:

An initial look at this PE indicated it has a high entropy, so this binary is very likely to be packed.

file	
file > sha256	76DEC74126E2B70D47786720BC172B86D1B15616DA3ABC09AEBA6035D95C0BF7
file > first 32 bytes (hex)	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00
file > first 32 bytes (text)	MZ.....@.....
file > info	size: 10240 bytes, entropy: 7.331
file > type	executable, 64-bit, GUI
file > version	n/a
file > description	n/a
entry-point > first 32 bytes (hex)	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00
entry-point > location	0x0000C6F0
file > signature	Microsoft Linker 2.44

property	value	value	value
section	section[0]	section[1]	section[2]
name	.text	.data	.rdata
section > sha256	n/a	D80845C1C11469597DF4C49...	C9B3267D2A2B2F1B05FB7C...
entropy	n/a	7.622	2.222
file > ratio (95.00%)	n/a	90.00 %	5.00 %
raw-address (begin)	0x00000200	0x00000200	0x00002600
raw-address (end)	0x00000200	0x00002600	0x00002800
raw-size (9728 bytes)	0x00000000 (0 bytes)	0x00002400 (9216 bytes)	0x00000200 (512 bytes)
virtual-address (begin)	0x00001000	0x0000B000	0x0000E000
virtual-address (end)	0x0000B000	0x0000E000	0x0000F000
virtual-size (57344 bytes)	0x0000A000 (40960 bytes)	0x00003000 (12288 bytes)	0x00001000 (4096 bytes)
characteristics	0xE0000080	0xE0000040	0xC0000040
write	x	x	x
execute	x	x	-
share	-	-	-
self-modifying	x	x	-
virtual	x	-	-

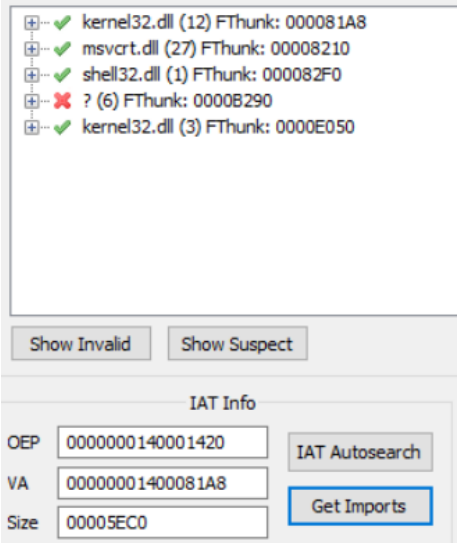
To manually unpack this sample, x64DBG can be used. From PeStudio's analysis, there is an empty .text section that has permission to execute and is self-modifying, indicating that this is likely where the malware unpacks then runs. For this walkthrough, I disabled ASLR with CFF Explorer so you can follow along.



We can skip where it starts running this unpacked binary by looking at the end of the .data section in memory and look for a jump or call to a different region. Here, there's a jump to 0x1420 thats a strong candidate.



Setting a hardware breakpoint at this location (Right Click -> Breakpoint -> Set hardware on execution) will pause when we start executing this. Dumping this binary with Scylla requires a bit more work than normal now as it finds 2 kernel32.dlls and some invalid entries. Looking at the location of the VA it found shows that the size is much smaller than Scylla automatically guesses, with a real size closer to 0x150. Dumping now with a size of 150 and fixing the dump will give a functioning dumped binary now.



Now that we've got the unpacked binary, it can be loaded into Ghidra and we can locate known behavior (where its opening the youtube link). Looking at this function, there's a portion after it opens the link where it does some interesting stuff with memory and a few variables.

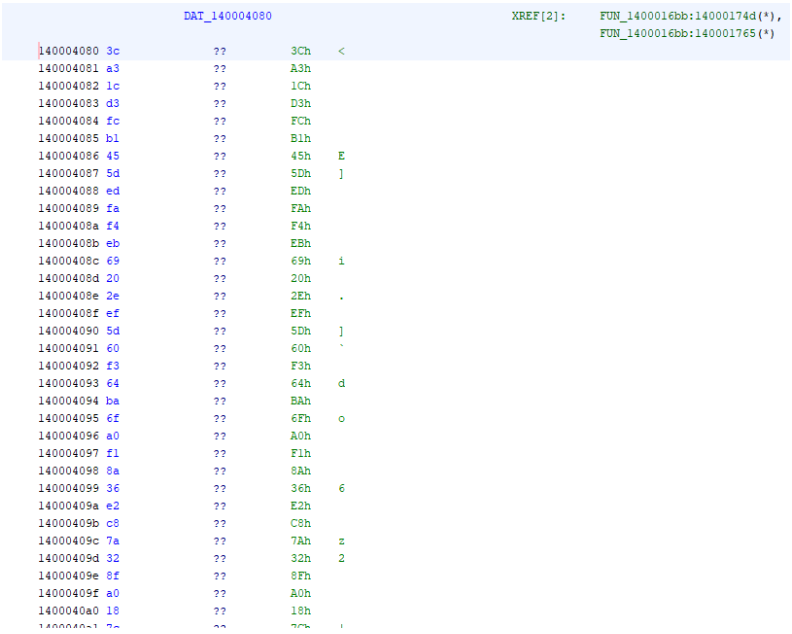
```
undefined8 FUN_1400016bb(void)

{
  undefined local_68 [73];
  undefined7 local_1f;
  undefined4 uStack_18;
  int local_14;
  int local_10;
  int local_c;

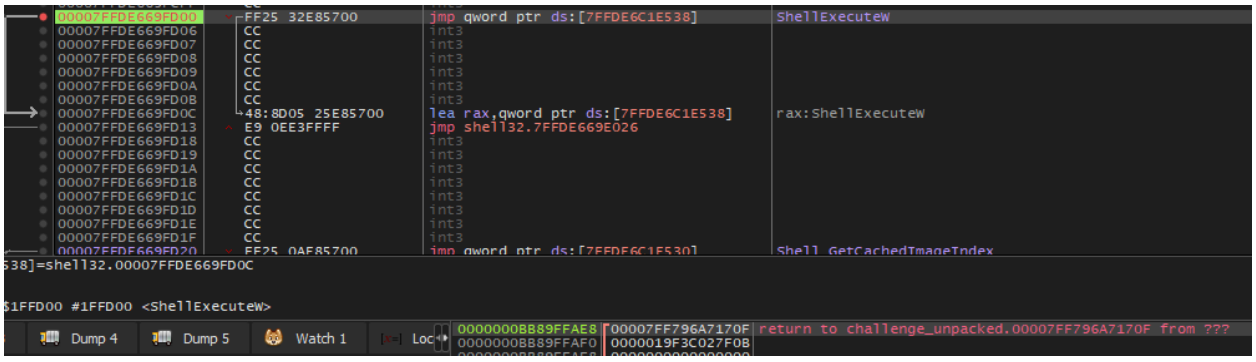
  ShellExecuteW((HWND)0x0,L"open",L"https://www.youtube.com/watch?v=L8XbI9aJOXk",(LPCWSTR)0x0,
                (LPCWSTR)0x0,1);
  local_1f = 0x6139496258384c;
  uStack_18 = 0x6b584f4a;
  local_10 = 0xb;
  local_14 = 0x23;
  memset(local_68,0,0x40);
  FUN_140001490((longlong)&local_1f,local_10,(longlong)&DAT_140004080,(longlong)local_68,local_14);
  for (local_c = 0; local_c < local_14; local_c = local_c + 1) {
  }
  return 0;
}

```

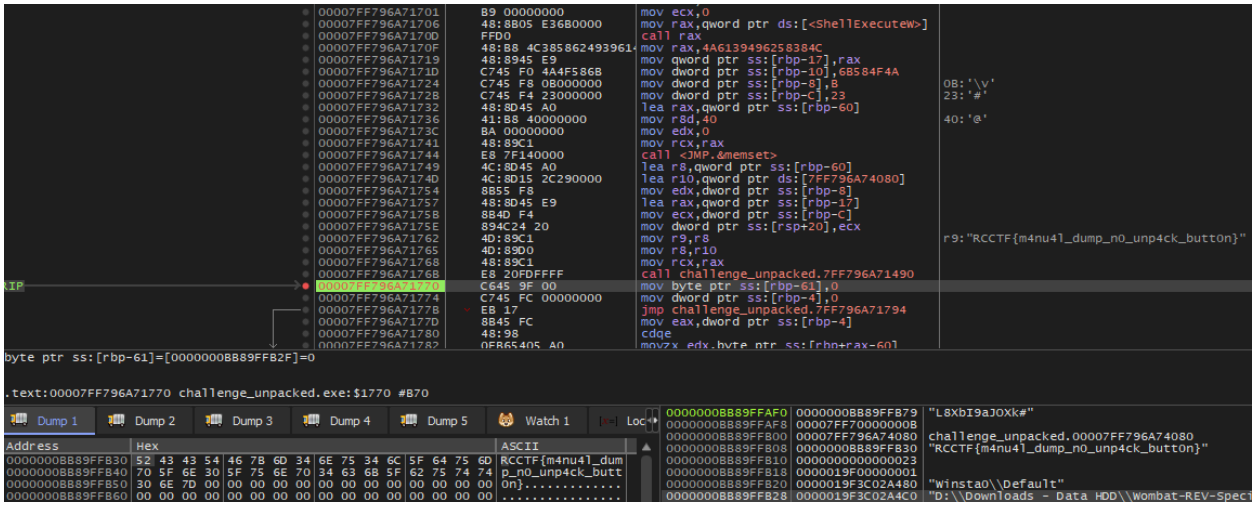
Looking at the data section it references, there's a bunch of bytes which seem pretty random, indicating it could be encrypted data.



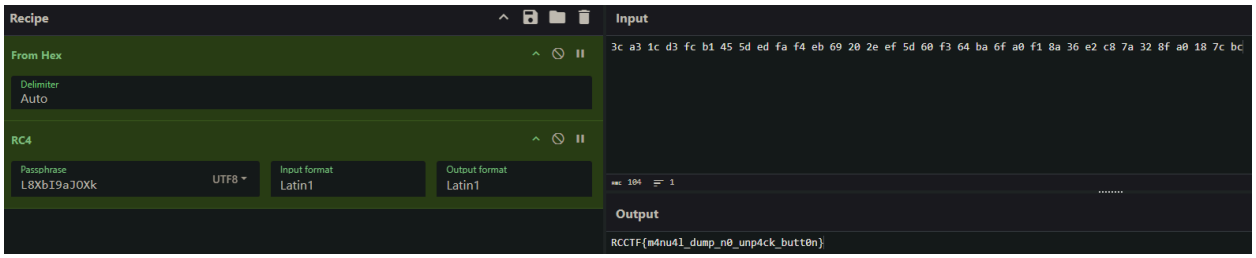
To make our lives easier, loading this into x64DBG will help us dynamically see whats going on with this memory region. Since we know ShellExecuteW will be called, `SetBPX ShellExecuteW` can be typed into the console to set a breakpoint there. Disabling other breakpoints then running until a breakpoint is hit will give a return address located on the stack to the function where this was called. I took these screenshots before I disabled ASLR, but you should see the same disassembly.



Locating the function and then using our knowledge of the functions' structure from Ghidra, setting a breakpoint after the second call should be after the data in memory is accessed. Running until that next breakpoint will reveal the flag in memory but x64DBG will also automatically present the value which is held in r9 at that point in the main CPU tab.



If instead you wanted to go down a fully static route, you might be able to discern that RC4 decryption is being performed on the data and the key is the youtube video ID.



Flag:

RCCTF{m4nu4l_dump_n0_unp4ck_butt0n}